Author:Ganesh Babu G

SDTM Programming — Complete Reference

SAS® vs R (tidyverse / dplyr / sdtm.oak) — All Scenarios Covered

SAS® Procedure / Data Step | **R / tidyverse / dplyr** Function / Package | **sdtm.oak** pharmaverse SDTM package

sdtm.oak mapping algorithms (assign_no_ct / assign_ct / hardcode_ct / hardcode_no_ct / condition_add / assign_datetime) • Controlled Terminology per domain • ISO 8601 edge cases (partial / unknown dates) • SUPP-- domain creation & long-string splitting • EPOCH derivation (FDA-required) • Domain-by-domain derivations: DM AE LB VS CM EX DS MH IE FA • VISITNUM/VISIT standardisation • Trial Design Domains (TA TE TV TI TS) • SV 10-step derivation • RELREC • MedDRA / WHODrug coding • --PRESP / --OCCUR / --STAT • define.xml & Pinnacle21 • Full 40+ domain checklist

1. sdtm.oak — CORE MAPPING ALGORITHMS

Function / Algorithm	SAS® Equivalent	Purpose
`assign_no_ct( raw_dat, raw_var, tgt_var, id_vars )`	`data ae; set ae; aeterm = raw_aeterm; run;`	Map raw variable to SDTM WITHOUT controlled terminology. Free-text fields: AETERM, CMTRT, MHTERM, etc.
`assign_ct( raw_dat, raw_var, tgt_var, ct_spec, ct_clst, id_vars )`	`format sex $sexfmt.; sex = put(raw_sex,$sexfmt.);`	Map raw value WITH CT lookup. Requires NCI codelist code e.g.'C66731'. Use for: SEX, RACE, VSORRESU, CMDOSU, etc.
`hardcode_ct( raw_dat, raw_var, tgt_var, tgt_val, ct_spec, ct_clst )`	`/* Hardcode + validate in CT */ vstest = 'Systolic Blood Pressure';`	Assign a FIXED value WITH CT validation. Use for VSTESTCD, VSTEST when values are protocol-defined and must be CT-compliant.
`hardcode_no_ct( raw_dat, raw_var, tgt_var, tgt_val )`	`domain = 'AE'; studyid = 'STUDY01';`	Assign a FIXED value WITHOUT CT check. Use for: DOMAIN, STUDYID, flag constants.
`condition_add( raw_dat, condition_expr )`	`/* WHERE / IF filter before assign */ if mdprior = '1';`	Filter raw data BEFORE mapping. Returns conditioned data frame — used as raw_dat. Essential for conditional derivations.
`assign_datetime( raw_dat, raw_var = c('dt','tm'), tgt_var, raw_fmt, raw_unk = c('UN','UNK') )`	`aestdtc = catx('T', put(aedt,yymmdd10.), put(aetm,time8.));`	Convert raw date+time ISO 8601 DTC. Handles partial dates, unknown tokens (UNK '--'). Apply to ALL --DTC variables in every domain.
`derive_seq( sdtm_in, tgt_var = 'AESEQ' )`	`by usubjid; if first.usubjid then seq=0; seq+1;`	Derive --SEQ sequence number within USUBJID. Automatically groups by subject.
`derive_study_day( sdtm_in, dm_domain, tgt_var='AESTDY', ref_date='RFSTDTC', src_date='AESTDTC' )`	`aestdy = input(aestdtc,yymmdd10.) - input(rfstdtc,yymmdd10.) + (aestdtc>=rfstdtc);`	Derive --DY (study day) from DTC variable relative to reference date in DM.
`generate_sdtm_supp( sdtm_in, supp_qual_info )`	`/* Manual pivot + transpose to build SUPP structure */`	Build SUPP-- domain from a specification list. Handles IDVAR/IDVARVAL automatically. New in sdtm.oak v0.2+
`oak_id_vars()`	`/* BY key variables for record traceability */`	Standard ID vars linking SDTM records back to raw source records. Required argument in all mapping functions.

sdtm.oak v0.2.0 added DM domain support and generate_sdtm_supp(). Trial Design Domains, SV, SE, RELREC, and Associated Persons are not yet in sdtm.oak scope.

2. CONTROLLED TERMINOLOGY (CT) HANDLING

Task	SAS®	R / sdtm.oak
Load CT codelist file	`PROC IMPORT datafile='ct.xlsx' out=ct dbms=xlsx replace; run;`	`ct_spec <- read_ct_spec('ct.csv') # test data: read_ct_spec_example()`

Author:Ganesh Babu G

Task	SAS®	R / sdtm.oak
Map value using CT codelist (SEX — NCI C66731)	`proc format; value $sexfmt 'Male'='M' 'Female'='F'; run; sex = put(raw_sex,$sexfmt.);`	`assign_ct( raw_dat=dm_raw, raw_var='RAW_SEX', tgt_var='SEX', ct_spec=study_ct, ct_clst='C66731' )`
RACE (NCI C74457)	`format race $racefmt.;`	`assign_ct(raw_var='RACERAW', tgt_var='RACE', ct_clst='C74457')`
Handle in ORRES/STRESC/STRESN	`if index(lborres,'<')>0 then do; lbstresc = lborres; lbstresn = .; end; else do; lbstresn = input(lborres,best.); lbstresc = strip(put(lbstresn,best.)); end;`	`mutate( lbstresn = if_else( str_detect(lborres,'[<>]'), NA_real_, as.numeric(lborres)), lbstresc = if_else( str_detect(lborres,'[<>]'), lborres, as.character(lbstresn)) )`
Apply CT from metacore spec	`/* Format catalogs + PROC FORMAT */`	`metatools::create var from codelist( metacore_obj, dataset, var_name )`
Validate term in CT (auto-warn)	`/* Manual if-then check */`	`# assign_ct() / hardcode_ct() # warn automatically if tgt_val # not found in the codelist`

3. ISO 8601 DATE HANDLING — EDGE CASES

Scenario	SAS®	R / sdtm.oak
Full date + time DTC	`aestdtc = catx('T', put(aedt,yymmdd10.), put(aetm,time8.));`	`assign_datetime( raw_var=c('AEDT','AETM'), tgt_var='AESTDTC', raw_fmt=c('d-m-y','H:M') )`
Partial — month only (YYYY-MM)	`aestdtc = cats(substr(aestdtc,1,7));`	`assign_datetime( raw_var=c('AEYR','AEMO'), tgt_var='AESTDTC', raw_fmt=c('yyyy','mm') ) # '2023-05'`
Year only (YYYY)	`aestdtc = put(aeyr,4.);`	`assign_datetime( raw_var='AEYR', tgt_var='AESTDTC', raw_fmt='yyyy' # '2023' )`
Unknown day (UNK) ISO dash	`if aeday='UNK' then aestdtc=cats(aeyr,'-',aemo,'-','--');`	`assign_datetime( raw_var=c('AEDT','AETM'), tgt_var='AESTDTC', raw_unk=c('UN','UNK') ) # UNK '2023-05--'`
Numeric SAS date ISO DTC	`aestdtc = put(aestdt, yymmdd10.);`	`aestdtc = format(aestdt,'%Y-%m-%d')`
Truncate datetime to date only	`aestdtc = substr(aestdtm,1,10);`	`str_sub(aestdtm, 1, 10)`
--DY from DTC (study day)	`aestdy = input(aestdtc,yymmdd10.) - input(rfstdtc,yymmdd10.) + (aestdtc >= rfstdtc);`	`derive_study_day( sdtm_in=ae, dm_domain=dm, tgt_var='AESTDY', ref_date='RFSTDTC', src_date='AESTDTC' )`

4. SUPP-- SUPPLEMENTAL QUALIFIER DOMAINS

Task	SAS®	R / sdtm.oak
Create SUPPAE from extra AE variables	`data suppae; set ae(keep=studyid usubjid aeseq aenomcd aesrccode); rdomain='AE'; idvar='AESEQ'; idvarval=strip(put(aeseq,best.)); qnam='AENOMCD'; qlabel='Nominal Code'; qval=aenomcd; output; qnam='AESRCCODE'; qval=aesrccode; output; run;`	`supp_info <- data.frame( qnam_var=c('AENOMCD','AESRCCODE'), qlabel_var=c('Nominal Code', 'Source Code') ) generate_sdtm_supp( sdtm_in = ae, supp_qual_info = supp_info )`
Merge SUPP back to parent domain	`proc sort data=suppae; by usubjid idvarval; data ae_full; merge ae suppae; by usubjid; run;`	`ae |> left_join( suppae |> pivot_wider( names_from = qnam, values_from = qval), by = c('usubjid', 'aeseq' = 'idvarval') )`
Split long text >200 chars (MHTERM1, MHTERM2 in SUPPMH)	`if length(mhterm)>200 then do; mhterm1=substr(mhterm,1,200); mhterm2=substr(mhterm,201,200); end; /* QNAM = MHTERM1, MHTERM2 */`	`mutate( mhterm1 = str_sub(mhterm,1,200), mhterm2 = if_else(nchar(mhterm)>200, str_sub(mhterm,201,400), NA_character_) ) # then generate_sdtm_supp()`
Required SUPP-- structure	`/* STUDYID RDOMAIN USUBJID IDVAR IDVARVAL QNAM QLABEL QVAL QORIG QEVAL */`	`# Enforced by generate_sdtm_supp() # RDOMAIN = domain code e.g. 'AE' # IDVAR = 'AESEQ' # IDVARVAL= as.character(aeseq)`

5. DOMAIN-BY-DOMAIN PROGRAMMING

5a. DM — Demographics

Variable	SAS® Derivation	R Derivation
USUBJID	`usubjid = catx('-', studyid,siteid,subjid);`	`paste(studyid,siteid,subjid,sep='-')`
AGE (from BRTHDTC)	`age = int( (input(rfstdtc,yymmdd10.) -input(brthdtc,yymmdd10.)) /365.25);`	`floor(time_length( interval(ymd(brthdtc), ymd(rfstdtc)),'years'))`
SEX (CT C66731)	`format sex $sexfmt.;`	`assign_ct(raw_var='SEXRAW', tgt_var='SEX', ct_clst='C66731')`
RACE (CT C74457)	`format race $racefmt.;`	`assign_ct(raw_var='RACERAW', tgt_var='RACE', ct_clst='C74457')`
DTHFL	`if not missing(dthdtc) then dthfl='Y';`	`mutate(DTHFL = if_else( !is.na(DTHDTC),'Y',NA_character_))`
RFSTDTC / RFENDTC	`/* min/max EX or DS dates */`	`derive_dm_dtc( dm_domain=dm, ex_domain=ex ) # sdtm.oak v0.2+`
ARM / ACTARM	`arm=trtgrp; actarm=acttrtgrp;`	`mutate(ARM=trtgrp, ACTARM=acttrtgrp)`
ARMCD (from TA domain)	`proc sql; select armcd from ta where arm=dm.arm; quit;`	`left_join(dm, ta |> select(arm,armcd), by='arm')`

5b. AE — Adverse Events

Variable	SAS®	R / sdtm.oak
AETERM (verbatim)	`aeterm = raw_aeterm;`	`assign_no_ct(raw_var='RAW_AETERM', tgt_var='AETERM')`
AEDECOD / AEBODSYS (MedDRA coding)	`proc sql; select a.*, b.aedecod, b.aebodsys, b.aehlt, b.aehlgt, b.aesoc from ae a left join meddra b on upcase(a.aeterm)= upcase(b.aeterm); quit;`	`ae |> left_join( meddra |> mutate( aeterm=toupper(aeterm)), by=join_by(toupper(aeterm)==aeterm) )`
AESTDTC / AEENDTC	`aestdtc=put(aestdt,yymmdd10.);`	`assign_datetime( raw_var=c('AEDR','AETM'), tgt_var='AESTDTC', raw_fmt=c('d-m-y','H:M') )`
AESEQ	`by usubjid; if first.usubjid then aeseq=0; aeseq+1;`	`derive_seq(sdtm_in=ae, tgt_var='AESEQ')`
AESEV (CT C66769)	`format aesev $sevfmt.;`	`assign_ct(raw_var='SEV_RAW', tgt_var='AESEV', ct_clst='C66769')`
AESER (Y/N)	`if serious='Yes' then aeser='Y'; else aeser='N';`	`hardcode_no_ct( raw_dat=condition_add( ae_raw, SERIOUS=='Yes'), tgt_var='AESER', tgt_val='Y' )`
AEACN (CT C66767)	`format aeacn $acnfmt.;`	`assign_ct(raw_var='ACNRAW', tgt_var='AEACN', ct_clst='C66767')`
EPOCH (FDA required)	`proc sql; select epoch from se where usubjid=ae.usubjid and aestdtc>=sestdtc and (aestdtc<=seendtc or seendtc=''); quit;`	`ae |> left_join( se |> select(usubjid,epoch, sestdtc,seendtc), by='usubjid') |> filter(aestdtc>=sestdtc, is.na(seendtc)|aestdtc<=seendtc)`

5c. LB — Laboratory (Findings class)

Variable	SAS®	R / sdtm.oak
LBTESTCD / LBTEST	`lbtestcd=labcd; lbtest=labnm;`	`hardcode_ct(raw_var='LABCD', tgt_var='LBTESTCD', tgt_val='ALT', ct_clst='C65047')`
LBORRES / LBORRESU	`lborres=strip(put(labval,best.)); lborresu=labunit;`	`assign_no_ct(raw_var='LABVAL', tgt_var='LBORRES')` `assign_ct(raw_var='LABUNIT', tgt_var='LBORRESU', ct_clst='C71620')`
LBSTRESC/LBSTRESN (handle < / > signs)	`if index(lborres,'<')>0 or index(lborres,'>')>0 then do; lbstresc=lborres; lbstresn=.; end; else do; lbstresn=input(lborres,best.); lbstresc=strip(put(lbstresn,best.)); end;`	`mutate( lbstresn=if_else( str_detect(lborres,'[<>]'), NA_real_, as.numeric(lborres)), lbstresc=if_else( str_detect(lborres,'[<>]'), lborres, as.character(lbstresn)) )`

Variable	SAS®	R / sdtm.oak
LBNRLO / LBNRHI	`lbnrlo=reflo; lbnrhi=refhi;`	`assign_no_ct(raw_var='REFLO', tgt_var='LBNRLO')`
LBNRIND (normal range)	`if lbstresnlbnrhi then lbnrind='HIGH'; else lbnrind='NORMAL';`	`mutate(lbnrind=case_when( lbstresnlbnrhi ~ 'HIGH', TRUE ~ 'NORMAL' ))`
LBBLFL (baseline flag)	`if last.lbtestcd and lbdtc<=rfstdtc then lbblfl='Y';`	`derive_blfl( sdtm_in=lb, tgt_var='LBBLFL', baseline_visits= c('SCREENING','BASELINE') ) # sdtm.oak v0.2+`

5d. VS — Vital Signs (one CRF row multiple SDTM records)

Step	SAS®	R / sdtm.oak
1. Pivot wide long per TESTCD	`proc transpose data=vs_raw out=vs_long (rename=(_name_=vstestcd col1=vsorres)); by usubjid visitnum; run;`	`vs_long <- vs_raw |> pivot_longer( cols=c(SYSBP,DIABP, PULSE,TEMP), names_to='vstestcd', values_to='vsorres' )`
2. VSTESTCD / VSTEST (CT C66741)	`if vstestcd='SYSBP' then vstest='Systolic BP';`	`vs_sysbp <- hardcode_ct( raw_dat=vs_raw, raw_var='SYS_BP', tgt_var='VSTESTCD', tgt_val='SYSBP', ct_clst='C66741' )`
3. VSORRESU / VSSTRESU (CT C66770)	`vsorresu='mmHg';`	`hardcode_ct(raw_var='SYS_BP', tgt_var='VSORRESU', tgt_val='mmHg', ct_clst='C66770')`
4. Combine all topic records	`data vs; set vs_sysbp vs_diabp vs_pulse vs_temp; run;`	`vs <- bind_rows( vs_sysbp, vs_diabp, vs_pulse, vs_temp )`
5. VSDTC (ISO 8601)	`vsdtc=put(vsdt,yymmdd10.);`	`assign_datetime( raw_var=c('VSDT','VSTM'), tgt_var='VSDTC', raw_fmt=c('d-m-y','H:M') )`
6. VSPOS (body position CT C71148)	`format vspos $posfmt.;`	`assign_ct(raw_var='POSRAW', tgt_var='VSPOS', ct_clst='C71148')`

5e. CM — Concomitant Medications

Variable	SAS®	R / sdtm.oak
CMTRT (verbatim)	`cmtrt = raw_mdterm;`	`assign_no_ct(raw_var='MDRAW', tgt_var='CMTRT')`
CMDECOD / CMCLAS (WHODrug coding)	`proc sql; select cmdecod,cmclas,cmclascd, cmatc,cmatcd from whodrug where upcase(cmtrt)=upcase(cm.cmtrt); quit;`	`cm |> left_join( whodrug |> mutate( cmtrt=toupper(cmtrt)), by=join_by(toupper(cmtrt)==cmtrt) )`
CMSTRTPT (ongoing prior: if MDPRIOR=1 BEFORE)	`if mdprior='1' then cmstrtpt='BEFORE';`	`hardcode_ct( raw_dat=condition_add( cm_raw, MDPRIOR=='1'), tgt_var='CMSTRTPT', tgt_val='BEFORE', ct_clst='C66728' )`
CMENRTPT (ongoing at end: if MDONG=1 ONGOING)	`if mdong='1' then cmenrtpt='ONGOING';`	`hardcode_ct( raw_dat=condition_add( cm_raw, MDONG=='1'), tgt_var='CMENRTPT', tgt_val='ONGOING', ct_clst='C66728' )`
CMROUTE (CT C66729)	`format cmroute $routefmt.;`	`assign_ct(raw_var='ROUTERAW', tgt_var='CMROUTE', ct_clst='C66729')`

5f. EX — Exposure

Variable	SAS®	R / sdtm.oak
EXTRT / EXDOSE	`extrt=drug; exdose=dose;`	`assign_no_ct(raw_var='DRUG',tgt_var='EXTRT')` `assign_no_ct(raw_var='DOSE',tgt_var='EXDOSE')`
EXDOSU (CT C71620)	`format exdosu $unitfmt.;`	`assign_ct(raw_var='DOSEUNIT', tgt_var='EXDOSU', ct_clst='C71620')`
EXSTDTC / EXENDTC	`exstdtc=put(exstdt,yymmdd10.);`	`assign_datetime( raw_var=c('EXSTDR','EXSTM'), tgt_var='EXSTDTC', raw_fmt=c('d-m-y','H:M') )`
EXSEQ	`by usubjid; if first.usubjid then exseq=0; exseq+1;`	`derive_seq(sdtm_in=ex, tgt_var='EXSEQ')`
EPOCH (from SE domain)	`/* lookup from SE domain */`	`left_join(ex, se|>select(usubjid,epoch, sestdtc,seendtc), by='usubjid') |> filter(exstdtc>=sestdtc)`

5g. DS / MH / IE / FA

Domain / Variable	SAS®	R / sdtm.oak
DS: DSDECOD (CT C66727)	`format dsdecod $dsdecfmt.;`	`assign_ct(raw_var='DISPRAW', tgt_var='DSDECOD', ct_clst='C66727')`
DS: DSTERM / DSSTDTC	`dsterm=dispreas; dsstdtc=put(dissdt,yymmdd10.);`	`assign_no_ct(raw_var='DISPREAS', tgt_var='DSTERM')` `assign_datetime(raw_var='DISSDT', tgt_var='DSSTDTC')`
MH: MHTERM / MHDECOD (MedDRA coding)	`mhterm=raw_mhterm; /* merge MedDRA coding file */`	`assign_no_ct(raw_var='HISTTERM', tgt_var='MHTERM') # MedDRA: left_join coding file`
MH: MHSTDTC (partial year/month OK)	`/* year only */ mhstdtc=put(mhyr,4.);`	`assign_datetime( raw_var=c('MHYR','MHMO'), tgt_var='MHSTDTC', raw_fmt=c('yyyy','mm') )`
IE: IETESTCD / IETEST (from TI domain)	`proc sql; select ietestcd,ietest from ti where ietest=ie.ietest; quit;`	`ie |> left_join( ti|>select(ietestcd,ietest), by='ietest' )`
FA: --PRESP / --OCCUR / --STAT	`if fapresp='Y' and faorres='' then fastat='NOT DONE';`	`mutate( fastat=if_else( fapresp=='Y' & is.na(faorres), 'NOT DONE', NA_character_) )`

6. EPOCH DERIVATION (FDA-required for all observation domains)

Task	SAS®	R dplyr
Derive EPOCH from SE domain (single domain)	`proc sql; create table ae_epoch as select a.*, b.epoch from ae a left join se b on a.usubjid=b.usubjid and a.aestdtc >= b.sestdtc and (a.aestdtc <= b.seendtc or b.seendtc=''); quit;`	`ae_epoch <- ae |> left_join( se|>select(usubjid,epoch, sestdtc,seendtc), by='usubjid') |> filter( aestdtc >= sestdtc, is.na(seendtc) | aestdtc <= seendtc ) |> select(-sestdtc,-seendtc)`
Add EPOCH across all domains (reusable function)	`/* Repeat PROC SQL per domain */`	`add_epoch <- function(df, dtc_var) { df |> left_join(se|>select(usubjid, epoch,sestdtc,seendtc), by='usubjid') |> filter(.data[[dtc_var]]>=sestdtc, is.na(seendtc)| .data[[dtc_var]]<=seendtc) |> select(-sestdtc,-seendtc) } lb <- add_epoch(lb, 'LBDTC') vs <- add_epoch(vs, 'VSDTC')`
EPOCH for ongoing records (no SEENDTC)	`if seendtc='' and aestdtc >= sestdtc;`	`filter(aestdtc>=sestdtc, is.na(seendtc) | aestdtc<=seendtc)`

7. VISITNUM / VISIT STANDARDISATION

Task	SAS®	R dplyr
Assign VISITNUM from TV domain	`proc sql; create table ae as select a.*, b.visitnum from ae a left join tv b on a.visit=b.visit and a.arm=b.arm; quit;`	`ae |> left_join( tv|>select(visit,arm,visitnum), by=c('visit','arm') )`
Validate VISIT against TV (flag CRF form names)	`proc sql; select distinct visit from ae where visit not in (select visit from tv); quit;`	`ae |> anti_join( tv|>distinct(visit), by='visit' ) |> distinct(visit) # Must return 0 rows if valid`
Unscheduled visit VISITNUM	`if visit='UNSCHEDULED' then visitnum=99;`	`mutate(visitnum=if_else( visit=='UNSCHEDULED', 99L, visitnum))`
VISITDY (planned day from TV)	`proc sql; select tvstrl from tv where visit=ds.visit; quit;`	`left_join(ds, tv|>select(visit,tvstrl), by='visit')`

8. TRIAL DESIGN DOMAINS (TA / TE / TV / TI / TS)

Domain	Key Variables	SAS® Example	R Example
TA Trial Arms	ARMCD, ARM, ETCD, ELEMENT, TABRANCH, EPOCH, TAETORD	`data ta; arm='ARM A'; armcd='ARMA'; etcd='SCRN'; epoch='SCREENING'; taetord=1; output; run;`	`ta <- tibble( arm='ARM A',armcd='ARMA', etcd='SCRN', epoch='SCREENING', taetord=1 )`
TE Trial Elements	ETCD, ELEMENT, TESTRL, TEENRL	`data te; etcd='SCRN'; element='Screening'; testrl='VISIT EQ 1'; teenrl='VISIT EQ 2'; output; run;`	`te <- tibble( etcd='SCRN', element='Screening', testrl='VISIT EQ 1', teenrl='VISIT EQ 2' )`

Domain	Key Variables	SAS® Example	R Example
TV Trial Visits	VISITNUM, VISIT, ARMCD, TVSTRL, TVENRL, TVELT	`data tv; visitnum=1; visit='SCREENING'; armcd='ARMA'; tvstrl='DAY -14'; tvenrl='DAY -1'; output; run;`	`tv <- tibble( visitnum=1, visit='SCREENING', armcd='ARMA', tvstrl='DAY -14', tvenrl='DAY -1' )`
TI Trial Inc/Exc	IETESTCD, IETEST, IECAT, TIRL, TIVERS	`data ti; ietestcd='INCL01'; ietest='Age >= 18 years'; iecat='INCLUSION'; output; run;`	`ti <- tibble( ietestcd='INCL01', ietest='Age >= 18 years', iecat='INCLUSION' )`
TS Trial Summary	TSPARMCD, TSPARM, TSVAL, TSVALNF, TSSEQ	`data ts; tsparmcd='NARMS'; tsparm='Number of Arms'; tsval='2'; output; tsparmcd='SSTDTC'; tsval='2020-01-01'; output; run;`	`ts <- tibble( tsparmcd=c('NARMS','SSTDTC'), tsparm=c('Number of Arms', 'Study Start Date'), tsval=c('2','2020-01-01') )`

9. SUBJECT VISITS (SV) — 10-STEP DERIVATION

SV contains one record per subject per visit. SVSTDTC/SVENDTC are derived from all observation domains. This is one of the most complex SDTM derivations.

Step	SAS®	R dplyr
1. Union all DTC variables from all domains	`proc sql; select usubjid,visitnum, visit,aestdtc as dtc from ae union all select usubjid,visitnum, visit,lbdtc as dtc from lb union all select usubjid,visitnum, visit,vsdtc from vs; quit;`	`all_dtcs <- bind_rows( ae|>select(usubjid,visitnum, visit,dtc=aestdtc), lb|>select(usubjid,visitnum, visit,dtc=lbdtc), vs|>select(usubjid,visitnum, visit,dtc=vsdtc) )`
2. Remove unscheduled & missing visits	`where not missing(visitnum) and visit ne 'UNSCHEDULED';`	`filter(!is.na(visitnum), visit!='UNSCHEDULED')`
3. Keep full dates only (length = 10 chars)	`where length(dtc)=10;`	`filter(nchar(dtc)==10)`
4. Derive SVSTDTC (min) and SVENDTC (max)	`proc means data=all_dtcs noprint; by usubjid visitnum visit; var dtc; output out=sv min=svstdtc max=svendtc; run;`	`sv <- all_dtcs |> group_by(usubjid,visitnum,visit) |> summarise( svstdtc=min(dtc,na.rm=TRUE), svendtc=max(dtc,na.rm=TRUE), .groups='drop')`
5. Add STUDYID, DOMAIN, ARMCD from DM	`/* merge DM for ARMCD */ studyid='STUDY01'; domain='SV';`	`sv <- sv |> mutate(studyid='STUDY01', domain='SV') |> left_join(dm|>select(usubjid, armcd), by='usubjid')`
6. Add planned visit info from TV	`proc sql; select tvstrl,tvenrl from tv where visit=sv.visit and armcd=sv.armcd; quit;`	`sv |> left_join( tv|>select(visit,armcd, tvstrl,tvenrl), by=c('visit','armcd') )`
7. Sort & derive SVSEQ	`proc sort data=sv; by usubjid visitnum; run; data sv; set sv; by usubjid; if first.usubjid then svseq=0; svseq+1; run;`	`sv <- sv |> arrange(usubjid,visitnum) |> group_by(usubjid) |> mutate(svseq=row_number()) |> ungroup()`

10. RELREC — RELATED RECORDS DOMAIN

Task	SAS®	R dplyr
Create RELREC linking AE CM	`data relrec; set ae(keep=studyid usubjid aeseq); rdomain='AE'; idvar='AESEQ'; idvarval=strip(put(aeseq,best.)); reltype='ONE'; relid='REL01'; output; /* repeat for CM records */ run;`	`relrec <- bind_rows( ae|>select(studyid,usubjid,aeseq)|> mutate(rdomain='AE', idvar='AESEQ', idvarval=as.character(aeseq), reltype='ONE',relid='REL01'), cm|>select(studyid,usubjid,cmseq)|> mutate(rdomain='CM', idvar='CMSEQ', idvarval=as.character(cmseq), reltype='ONE',relid='REL01') )`
FA linked to AE parent domain	`rdomain='FA'; idvar='FASEQ'; relid=strip(faobj);`	`# RELREC with RDOMAIN='FA' # matched to RDOMAIN='AE' # linked by shared RELID`
Required RELREC variables	`/* STUDYID RDOMAIN USUBJID IDVAR IDVARVAL RELTYPE RELID */`	`# tibble must contain: # studyid, rdomain, usubjid, # idvar, idvarval, # reltype ('ONE'/'MANY'), # relid (shared group ID)`

11. MedDRA & WHODrug DICTIONARY CODING

Task	SAS®	R dplyr
Merge MedDRA coding to AE (PT / HLT / HLGT / SOC)	`proc sql; create table ae as select a.*, b.aedecod, b.aebodsys, b.aehlt, b.aehlgt, b.aesoc from ae a left join meddra_coded b on upcase(a.aeterm)= upcase(b.aeterm); quit;`	`ae <- ae |> left_join( meddra_coded |> mutate(aeterm=toupper(aeterm)), by=join_by( toupper(aeterm)==aeterm) )`
MedDRA hierarchy in AE	`/* AEDECOD = Preferred Term (PT) AEHLT = High Level Term AEHLGT = High Level Group AEBODSYS= System Organ Class AESOC = Primary SOC */`	`# All from coding vendor output: # aedecod (PT) # aehlt (HLT) # aehlgt (HLGT) # aebodsys (SOC) # aesoc (primary SOC)`
WHODrug coding for CM	`proc sql; select a.*, b.cmdecod,b.cmclas, b.cmclascd,b.cmatc,b.cmatcd from cm a left join whodrug b on upcase(a.cmtrt)= upcase(b.cmtrt); quit;`	`cm <- cm |> left_join( whodrug |> mutate(cmtrt=toupper(cmtrt)), by=join_by( toupper(cmtrt)==cmtrt) )`
Handle uncoded terms (AEDECOD fallback)	`if missing(aedecod) then aedecod=aeterm;`	`mutate(aedecod= coalesce(aedecod,aeterm))`

12. --PRESP / --OCCUR / --STAT (Findings About rules)

Variable	Rule	SAS®	R
--PRESP (Pre-Specified)	'Y' if item was pre-specified on CRF	`if prespec='YES' then fapresp='Y';`	`mutate(fapresp=if_else( prespec=='YES','Y', NA_character_))`
--OCCUR (Occurrence)	'Y'=event occurred 'N'=did not occur Only when --PRESP='Y'	`if faorres='Yes' then faoccur='Y'; else if faorres='No' then faoccur='N';`	`mutate(faoccur=case_when( faorres=='Yes'~'Y', faorres=='No' ~'N', TRUE~NA_character_))`
--STAT (Completion Status)	'NOT DONE' if pre-specified but not collected/performed	`if fapresp='Y' and faorres='' then fastat='NOT DONE';`	`mutate(fastat=if_else( fapresp=='Y' & is.na(faorres), 'NOT DONE',NA_character_))`
QS/RS Domains (Questionnaires)	Create record even if 'NOT DONE'. Add QSREASND.	`if qsorres='' then do; qsstat='NOT DONE'; qsreasnd= 'LOGICALLY SKIPPED'; end;`	`mutate( qsstat=if_else( is.na(qsorres), 'NOT DONE',NA_character_), qsreasnd=if_else( is.na(qsorres), 'LOGICALLY SKIPPED', NA_character_) )`

13. DEFINE.XML GENERATION & PINNACLE21 VALIDATION

Task	SAS®	R pharmaverse
Generate define.xml	`/* defineIT, Pinnacle21 Enterprise, or SAS macro-based tools */ %define_write(specs=specs);`	`library(defineR) create_define( spec_path='define_spec.xlsx', out_path='define.xml' )`
Apply metadata before XPT write	`data adsl(label='Subject Level'); attrib usubjid length=$40 label='Unique Subject ID'; set adsl; run;`	`adsl |> xportr_label(metacore_obj) |> xportr_type(metacore_obj) |> xportr_length(metacore_obj) |> xportr_order(metacore_obj) |> xportr_write('adsl.xpt')`
ORIGIN traceability per variable	`/* Annotated CRF PDF ORIGIN='CRF' in define.xml */`	`# metacore stores ORIGIN: # metacore_obj$var_spec$origin # Values: 'CRF','Derived', # 'Assigned','Protocol','eDT'`
Pinnacle21 conformance check	`/* Pinnacle21 Community / Enterprise — XPT + define.xml */`	`# 1. xportr_write() XPT # 2. defineR define.xml # 3. Upload to Pinnacle21 Community # 4. Review ERRORS/WARNINGS/NOTICES`
Unit testing / peer QC	`/* PROC COMPARE + log review */`	`testthat::test_that( 'SAFFL is Y or N', { expect_true( all(adsl$SAFFL %in% c('Y','N'))) })`

14. COMPLETE SDTM DOMAIN CHECKLIST (SDTMIG v3.4)

Class	Dom	Full Name	sdtm.oak v0.2	Key Variables
Special Purpose	**DM**	Demographics	Yes	USUBJID AGE SEX RACE RFSTDTC ARM ARMCD DTHFL
Special Purpose	**CO**	Comments	Yes	COVAL CODTC COREF
Special Purpose	**SE**	Subject Elements	Partial	ETCD EPOCH SESTDTC SEENDTC
Special Purpose	**SV**	Subject Visits	Not in scope	VISITNUM VISIT SVSTDTC SVENDTC ARMCD
Special Purpose	**SM**	Disease Milestone	Partial	MIDSTYPE SMTPT SMDTC
Events	**AE**	Adverse Events	Yes	AETERM AEDECOD AEBODSYS AESTDTC AEENDTC AESEV AESER EPOCH
Events	**DS**	Disposition	Yes	DSTERM DSDECOD DSSTDTC DSCAT
Events	**MH**	Medical History	Yes	MHTERM MHDECOD MHSTDTC MHENDTC
Events	**CE**	Clinical Events	Yes	CETERM CEDECOD CESTDTC CEENDTC
Events	**HO**	Healthcare Encounters	Yes	HOTERM HOTYPE HOSTDTC HOENDTC
Interventions	**CM**	Concomitant Meds	Yes	CMTRT CMDECOD CMDOSE CMROUTE CMSTDTC CMENRTPT
Interventions	**EX**	Exposure	Yes	EXTRT EXDOSE EXDOSU EXSTDTC EXENDTC EPOCH
Interventions	**EC**	Exposure as Collected	Yes	ECTRT ECDOSE ECSTDTC ECENDTC
Interventions	**SU**	Substance Use	Yes	SUTRT SUCAT SUSTDTC SUENDTC
Interventions	**PR**	Procedures	Yes	PRTRT PRSTDTC PRENDTC
Findings	**EG**	ECG	Yes	EGTESTCD EGORRES EGSTRESC EGSTRESN EGDTC
Findings	**IE**	Incl/Excl Criteria	Yes	IETESTCD IETEST IEORRES IESTDTC
Findings	**LB**	Laboratory	Yes	LBTESTCD LBORRES LBSTRESC LBSTRESN LBNRLO LBNRHI LBNRIND
Findings	**MB**	Microbiology	Yes	MBTESTCD MBORRES MBSPEC MBDTC
Findings	**PC**	PK Concentration	Yes	PCTESTCD PCORRES PCSTRESC PCSTRESN PCDTC PCSPEC
Findings	**PP**	PK Parameters	Yes	PPTESTCD PPORRES PPSTRESC PPSTRESN
Findings	**QS**	Questionnaires	Yes	QSTESTCD QSORRES QSSTRESC QSDTC QSSTAT QSREASND
Findings	**RS**	Disease Response	Yes	RSTESTCD RSORRES RSDTC RSEVAL
Findings	**SC**	Subject Characteristics	Yes	SCTESTCD SCORRES SCDTC
Findings	**TR**	Tumor Results	Yes	TRTESTCD TRORRES TRDTC VISIT TRGRPID
Findings	**TU**	Tumor Identification	Yes	TULOC TUMETHOD TUDTC TUGRPID
Findings	**VS**	Vital Signs	Yes	VSTESTCD VSORRES VSSTRESC VSDTC VSPOS EPOCH
Findings About	**FA**	Findings About	Yes	FAOBJ FATESTCD FAORRES FAPRESP FAOCCUR FASTAT
Trial Design	**TA**	Trial Arms	Not in scope	ARMCD ARM ETCD ELEMENT TABRANCH EPOCH TAETORD
Trial Design	**TE**	Trial Elements	Not in scope	ETCD ELEMENT TESTRL TEENRL
Trial Design	**TV**	Trial Visits	Not in scope	VISITNUM VISIT ARMCD TVSTRL TVENRL TVELT
Trial Design	**TI**	Trial Inc/Exc	Not in scope	IETESTCD IETEST IECAT TIRL TIVERS
Trial Design	**TS**	Trial Summary	Not in scope	TSPARMCD TSPARM TSVAL TSVALNF TSSEQ
Relationship	**RELREC**	Related Records	Not in scope	RDOMAIN IDVAR IDVARVAL RELTYPE RELID
SUPP	**SUPP--**	Supplemental Qualifiers	Yes (v0.2+)	RDOMAIN IDVAR IDVARVAL QNAM QLABEL QVAL QORIG

References: SDTMIG v3.4 (CDISC) • sdtm.oak v0.2.0 (pharmaverse) • FDA Study Data Technical Conformance Guide v5.5 • MedDRA v27 • WHODrug Global • Pinnacle21